Open in Colab

Prelab 2.2 Signal processing, visualization, time and frequency domain practice

2.1 Sample signal processing and visualization practice

For practicing signal processing and visualization, let's first create signals. The sample code for generating and visualization of signals is below Figure 5 as Python - Python3 (Signal plotting code). The scenario is that sampling frequency (f_s), sampling period (T), and number of samples (N) are 100 Hz, 0.01 sec-1, and 100, respectively. Therefore, total length of the signal is 100 for 1 second. The sample signal (x) are combinations of three signals (x_1, x_2, and x_{noise}). The first signal, x_1, is sine wave signal with 10 Hz main frequency and amplitude 2. The second signal, x_2, is cosine wave signal with 15 Hz main frequency and amplitude 1.5. The third signal, x_{noise}, is random signal of normal distribution from 0 to 0.3. By summing three signals, x can be defined as Eq. (1).

(1):

$$x[n] = x_1[n] + x_2[n] + x_{noise}[n]$$

---

where n is sample index. By using Matplotlib library of Python, signals in time domain are plotted as Figure 4 and Figure 5. Run the sample code and check the plots.

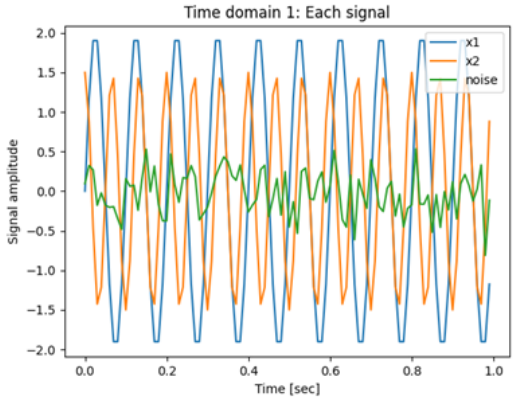


Figure 4 Time domain plot 1 of each signal

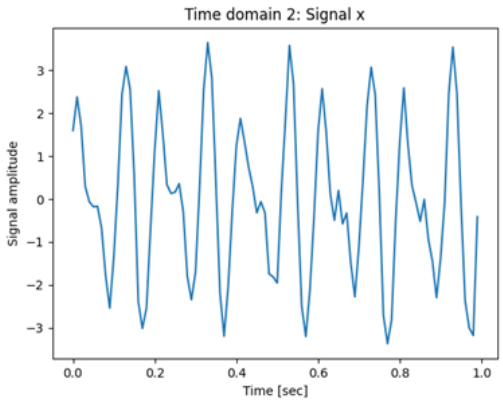


Figure 5 Time domain plot 2 of combined signal

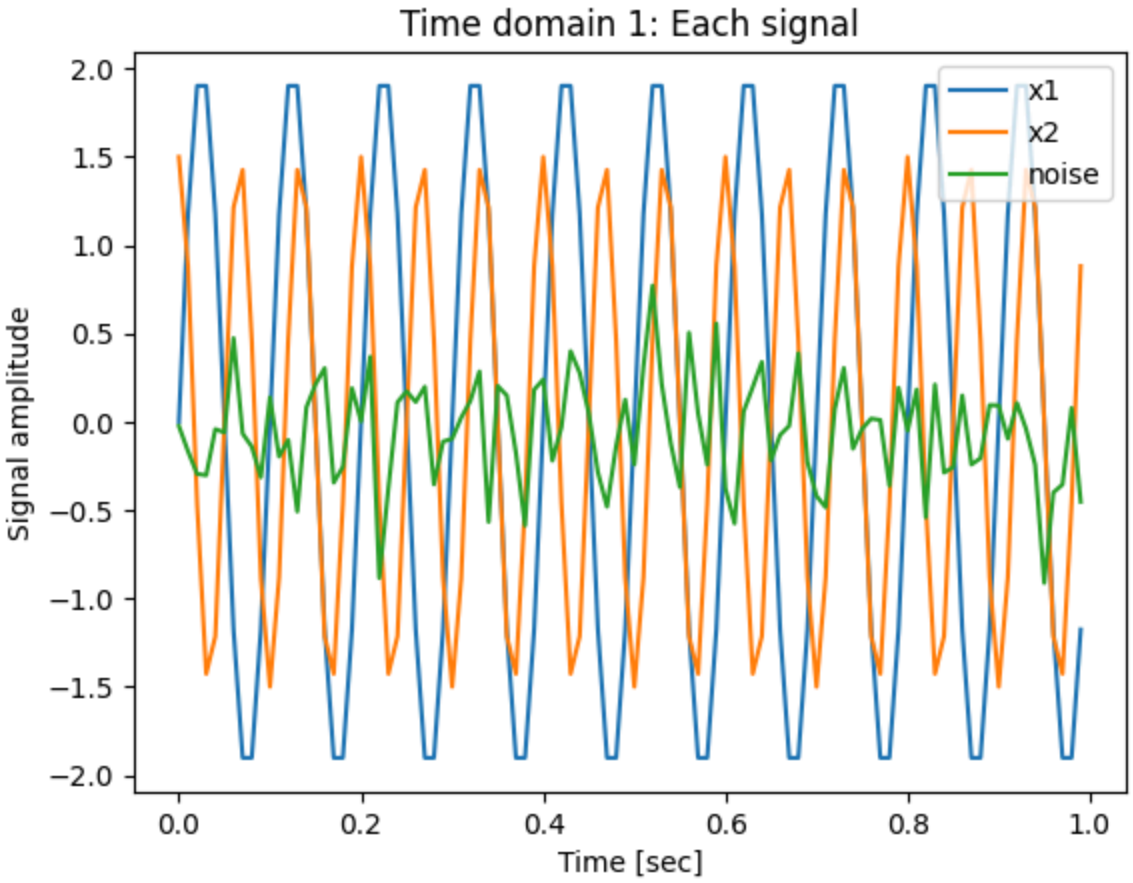
Python - Python3 (Signal plotting code)

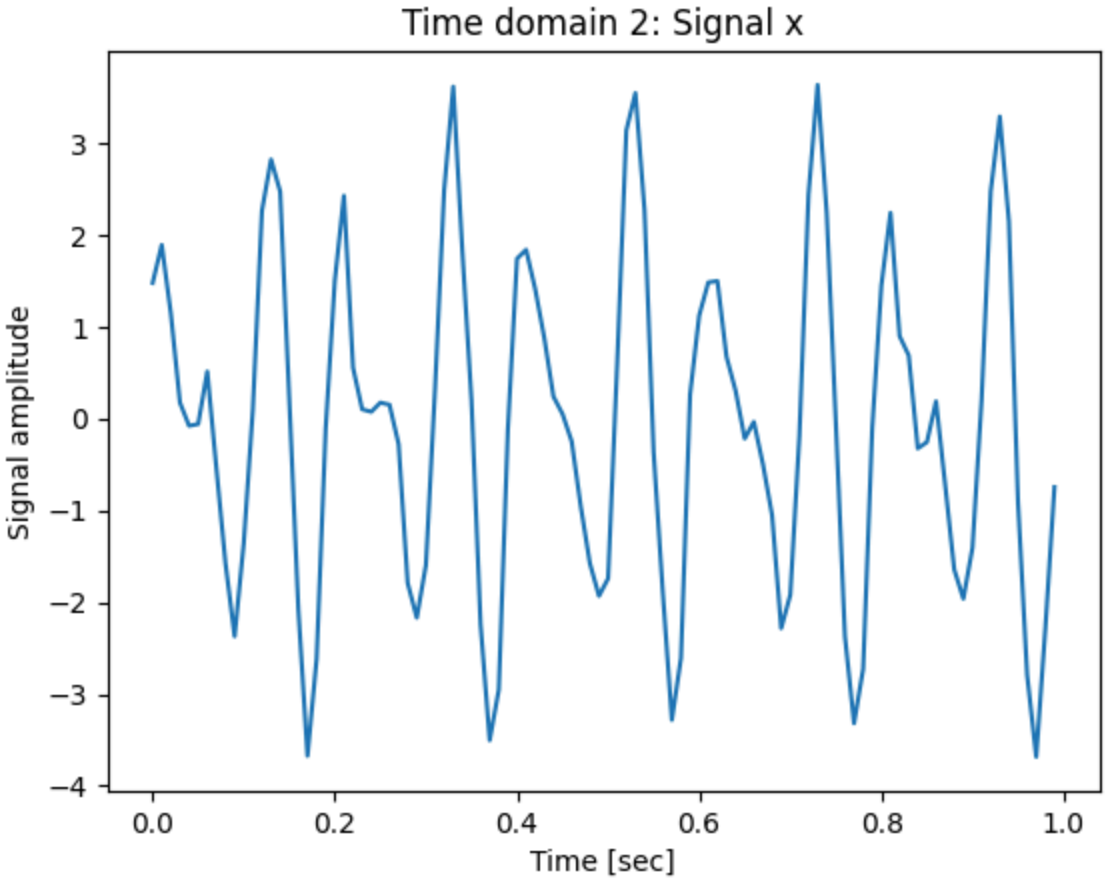
Please run the code below in Google Colab to practice plotting the visualization.

In [11]:

```
import numpy as np
import matplotlib
#matplotlib.use('tkagg')
import matplotlib.pyplot as plt

fs = 100 # sampling frequency, fs = 100 Hz
T = 1/fs # sampling period, T = 1/100 [sec^-1]
N = 100 # number of samples

f1 = 10 # main frequency 1, f1 = 10 Hz
f2 = 15 # main frequency 2, f2 = 15 Hz

t = np.linspace(0, N*T, N, endpoint=False) # define time vector, t
x1 = 2 * np.sin(f1 * 2 * np.pi * t) # signal 1, amplitude = 2, frequency = 1
x2 = 1.5 * np.cos(f2 * 2 * np.pi * t) # signal 2, amplidue = 1.5, frequency
noise = np.random.normal(0,0.3,N) # noise, random value normal distribution
x =  x1 + x2 + noise # define signal vector, x

# plot each signal
fig1 = plt.figure(1)
ax1 = fig1.gca()
ax1.plot(t, x1)
ax1.plot(t, x2)
ax1.plot(t, noise)
ax1.legend(["x1", "x2", "noise"])
ax1.set_xlabel("Time [sec]")
ax1.set_ylabel("Signal amplitude")
ax1.set_title("Time domain 1: Each signal")

# plot summation of all signal, output signal x
fig2 = plt.figure(2)
ax2 = fig2.gca()
ax2.plot(t, x)
ax2.set_xlabel("Time [sec]")
```

```
ax2.set_ylabel("Signal amplitude")
ax2.set_title("Time domain 2: Signal x")

plt.show()
```





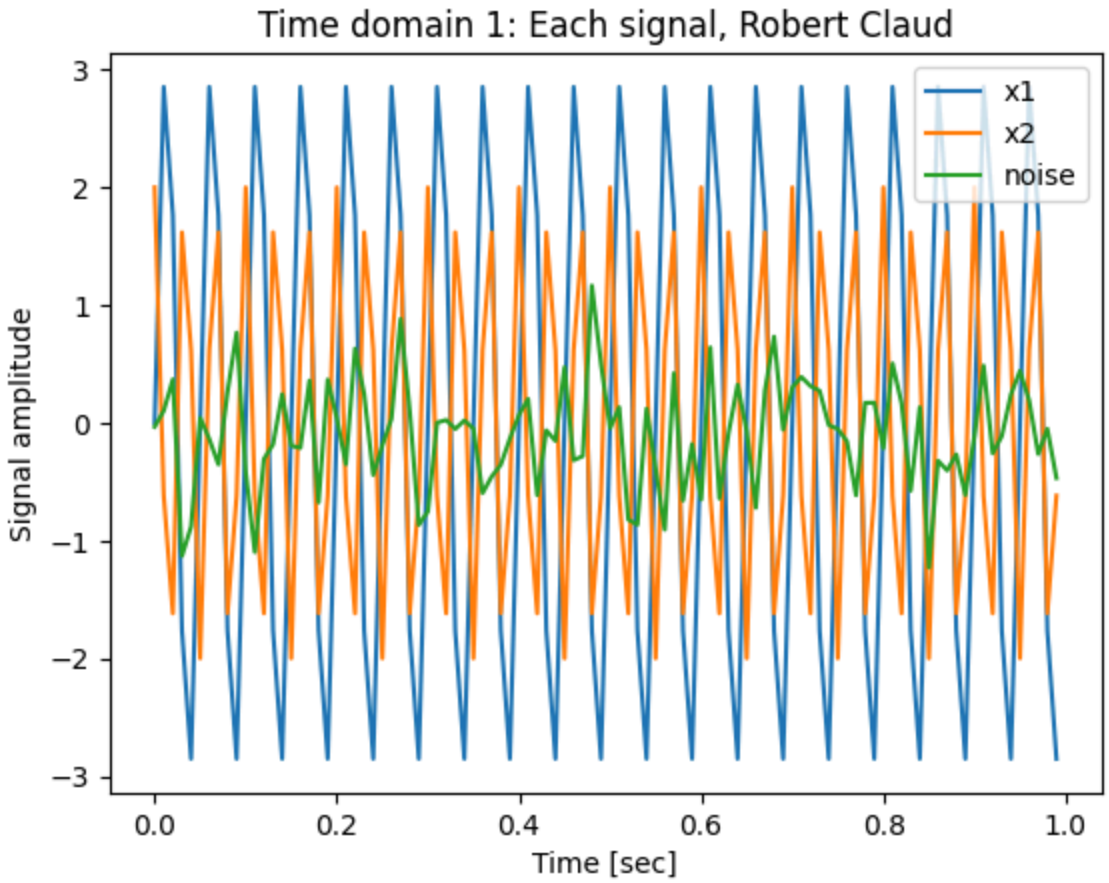
Task 2.1

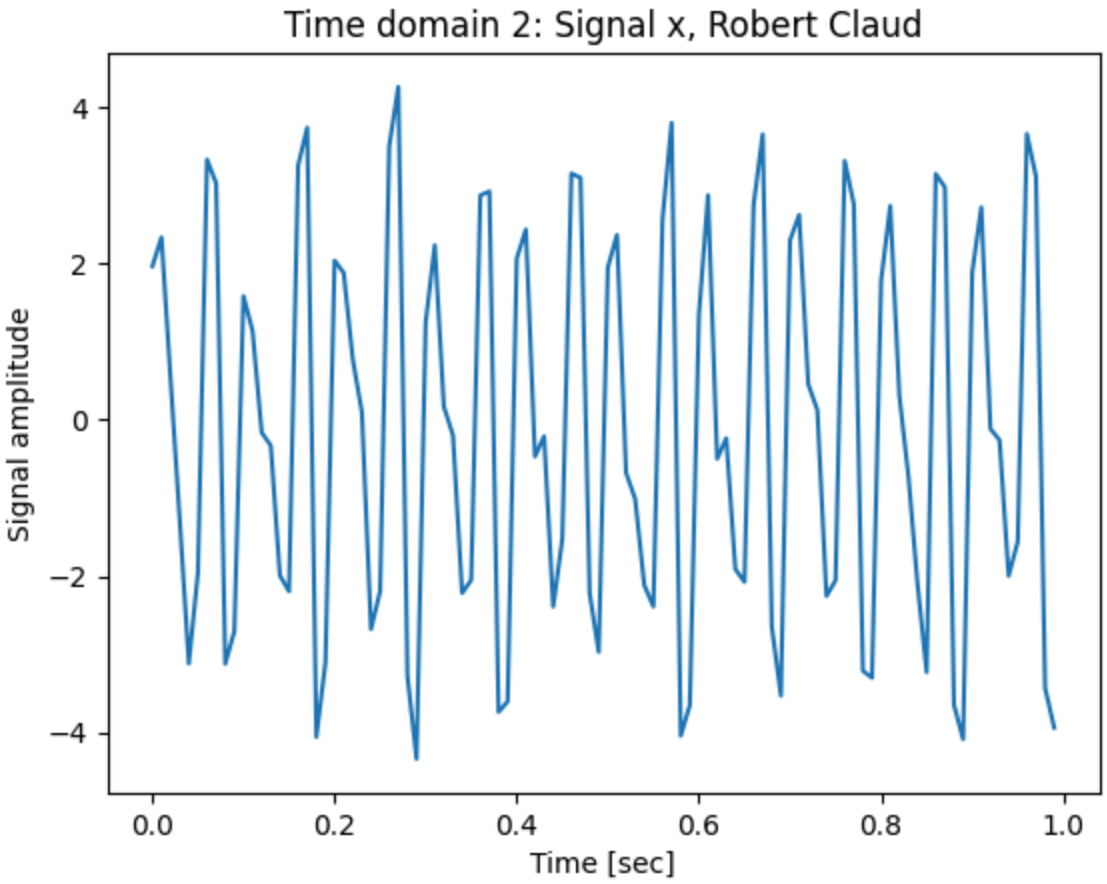
Modify the code above and experiment with the different parameters given below. In the Code Cell below, copy and paste the code, modify the parameters, and you can run the plt.show() function to plot the graphs.

1. Change amplitude and main frequency of signal 1, x_1, to 3 and 20 Hz, respectively.
2. Change amplitude and main frequency of signal 2, x_2, to 2 and 30 Hz, respectively.
3. Change random noise signal, x_{noise}, distribution from 0 to 0.5.
4. Add your name at the end of the title of each plot (e.g., 'Time domain 1: Each signal, John Doe')

In [12]:
```
#Write your code here.
import numpy as np
import matplotlib
#matplotlib.use('tkagg')
import matplotlib.pyplot as plt

fs = 100 # sampling frequency, fs = 100 Hz
T = 1/fs # sampling period, T = 1/100 [sec^-1]
N = 100 # number of samples

f1 = 20 # main frequency 1, f1 = 20 Hz
f2 = 30 # main frequency 2, f2 = 30 Hz
```

```
t = np.linspace(0, N*T, N, endpoint=False) # define time vector, t
x1 = 3 * np.sin(f1 * 2 * np.pi * t) # signal 1, amplitude = 3, frequency = f
x2 = 2 * np.cos(f2 * 2 * np.pi * t) # signal 2, amplidue = 2, frequency = f2
noise = np.random.normal(0,0.5,N) # noise, random value normal distribution
x =  x1 + x2 + noise # define signal vector, x

# plot each signal
fig1 = plt.figure(1)
ax1 = fig1.gca()
ax1.plot(t, x1)
ax1.plot(t, x2)
ax1.plot(t, noise)
ax1.legend(["x1", "x2", "noise"])
ax1.set_xlabel("Time [sec]")
ax1.set_ylabel("Signal amplitude")
ax1.set_title("Time domain 1: Each signal, Robert Claud")

# plot summation of all signal, output signal x
fig2 = plt.figure(2)
ax2 = fig2.gca()
ax2.plot(t, x)
ax2.set_xlabel("Time [sec]")
ax2.set_ylabel("Signal amplitude")
ax2.set_title("Time domain 2: Signal x, Robert Claud")

plt.show()
```





2.2 Time domain

Let's try signal processing using Numpy and SciPy. First, time domain features are statistical analysis of data set. 7 selected features in time domain are shown in Table 1. These features are widely used for data analytics. Try to find the time domain features from the signal generated in Task 2.1.

Table 1 Time-domain feature[1]

Number	Feature	Equation
1	Mean	$x_{\mathrm{m}} = \frac{\sum_{n=1}^{N} x[n]}{N}$
2	Standard deviation	$x_{\mathrm{std}} = \sqrt{\frac{\sum_{n=1}^{N} (x[n] - x_{\mathrm{m}})^2}{N-1}}$
3	Root mean square	$x_{\mathrm{rms}} = \sqrt{\frac{\sum_{n=1}^{N} (x[n])^2}{N}}$
4	Peak	$x_{\mathrm{peak}} = \max\lvert x[n] \rvert$
5	Skewness	$x_{\mathrm{skew}} = \frac{\sum_{n=1}^{N} (x[n] - x_{\mathrm{m}})^3}{(N-1)x_{\mathrm{std}}^3}$
6	Kurtosis	$x_{\mathrm{kurt}} = \frac{\sum_{n=1}^{N} (x[n] - x_{\mathrm{m}})^4}{(N-1)x_{\mathrm{std}}^4}$
7	Crest factor	$x_{\mathrm{cf}} = \frac{x_{\mathrm{peak}}}{x_{\mathrm{rms}}}$

[1]Lei, Yaguo, et al. "New clustering algorithm-based fault diagnosis using compensation distance evaluation technique." Mechanical Systems and Signal Processing 22.2 (2008): 419-435.

Task 2.2

1. Using the code below, complete the formulas from Table 1 (above) using Python script.

2. Find time domain features by using the code below with the data stored in the mixed signal variable 'x'.

- Once you finish, you can execute them in this Colab notebook.
- As reminder, keep the changed values (amplitudes, main frequencies, and so on) from Task 2.1.

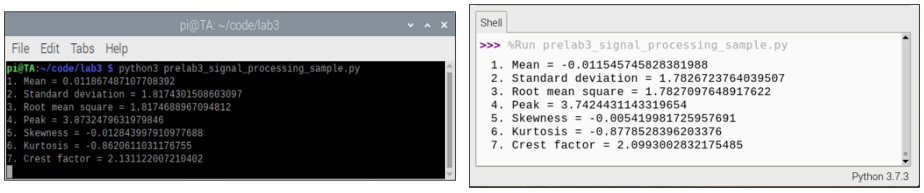


Figure 6 Time domain features of example signal: terminal window (left) and Thonny shell (right)

In [13]:
```
from scipy.stats import skew # package used to calculate skewness
from scipy.stats import kurtosis # package used to calculate kurtosis
import numpy as np # numpy can be used to calculate the necessary values bel

# Recall that data is saved in the signal variables: x1, x2, and x

# Modify and write your code here to calculate these time domain features.

#The first two metrics has been calculated. You can either complete the rest

#As you complete each metric, you can 'uncomment' each line to be executed a

x_m = sum(x)/len(x) # mean
x_std = np.sqrt(sum([(y - x_m) ** 2 for y in x])/(len(x)-1)) # standard
x_rms = np.sqrt(np.sum([(y ** 2) for y in x])/len((x)-1)) # rms
x_peak = np.max([abs(y) for y in x]) # peak
x_skew = np.sum([(y - x_m) ** 3 for y in x]) / ((len((x)-1) * x_std ** 3)) #
x_kurt = np.sum([(y - x_m) ** 4 for y in x]) / ((len((x)-1) * x_std ** 4)) #
x_cf = x_peak / x_rms # crest factor


print("1. Mean =", x_m)
print("2. Standard deviation =", x_std)
print("3. Root mean square =", x_rms)
print("4. Peak =", x_peak)
print("5. Skewness =", x_skew)
```

```
print("6. Kurtosis =", x_kurt)
print("7. Crest factor =", x_cf)
```

```
1. Mean = -0.09823042377065047
2. Standard deviation = 2.5645123770126474
3. Root mean square = 2.553547671512461
4. Peak = 4.337878526327046
5. Skewness = 0.008186181304550445
6. Kurtosis = 1.5371776018640615
7. Crest factor = 1.698765437090011
```

2.3 Frequency domain

Frequency domain features and analysis refer to performing FFT (Fast Fourier Transform). FFT is a mathematical expression and algorithm which computes DFT (Discrete Fourier Transform) of a data set, so-called Fourier analysis. It allows us to convert signals from the original domain, normally the time domain, to the frequency domain. The 1D (one-dimensional) DFT signal, y[k], of length-N sequence of signal from x[n] is defined as Eq. (2). The maximum frequency range of FFT from repetitive signals or oscillating systems is Nyquist frequency, $f_s/2$.

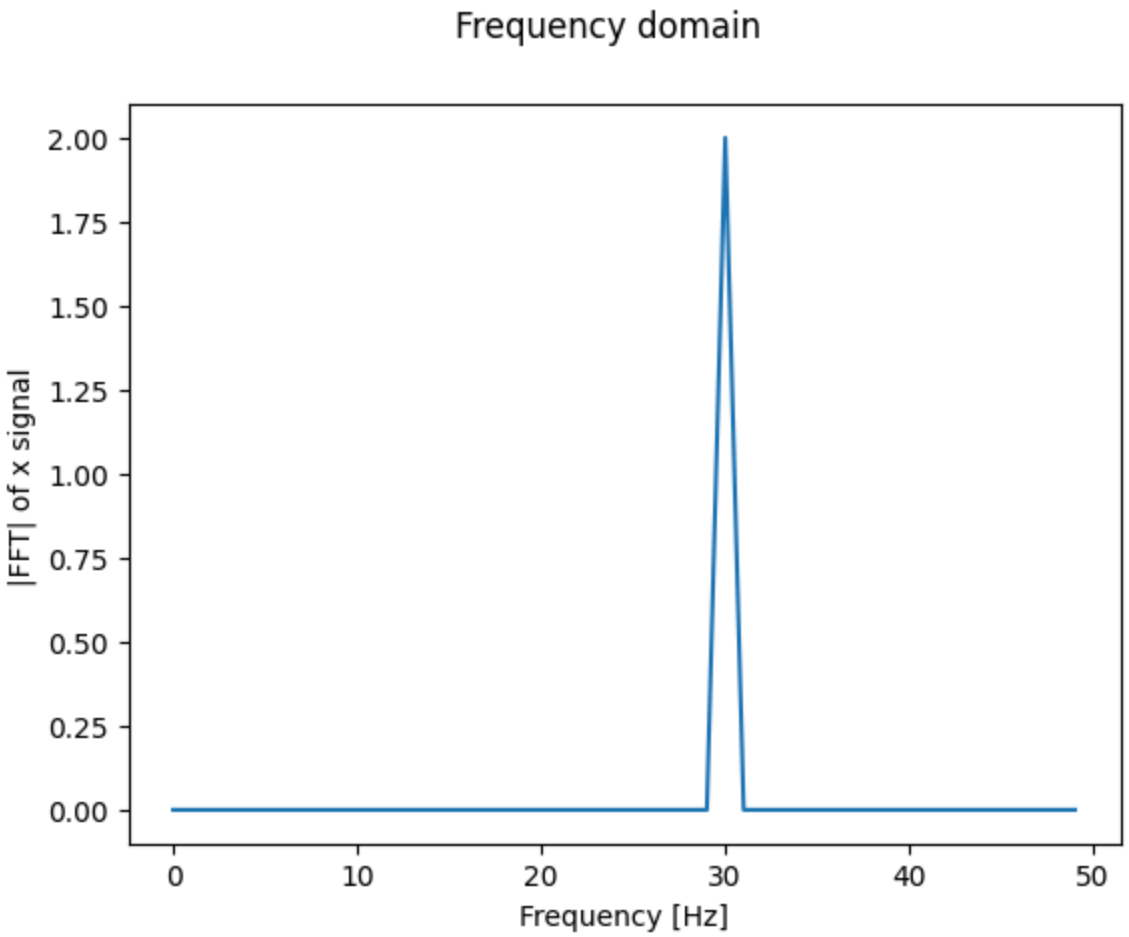
(2):

$$y[k] = \sum_{n=0}^{N-1} e^{-2\pi j \frac{kn}{N}} x[n]$$

FFT is a useful method to analyze data in the frequency domain especially for high-frequency sampling data as well as image processing. If you are interested in further study and principle of FFT and frequency domain analysis, it is recommended to take courses, MA 511 - Linear Algebra with Application and ME 579 – Fourier Methods in Digital Signal Processing. Other common frequency domain analysis methods are Laplace transform (control systems), Z transform (discrete-time processing), Wavelet transform (image processing), and so on. But in the future lab, we will only use FFT for frequency domain analysis.

In [14]:
```
#Here is a sample code to determine FFT for signal X2

#First, import fft libraries
from scipy.fft import fft, fftfreq

# get_fft is a function to calculate FFT
# Inputs:
# x = # (signal array)
# T (sampling period)
# N (number of Samples)

# Outputs:
# f (frequency array)
# y_mag (FFT magnitude array)
```

```
#FFT function.
def get_fft(x, T, N):
    f = fftfreq(N, T)[:N//2]
    y_mag = 2/N * np.abs(fft(x)[:N//2])
    y_mag[0] = 0 # Making 0 Hz frequency component zero
    return f, y_mag

#Output variables
f_x, y_x = get_fft(x2, T, N) #Signal x2 is been used

#Plotting fft vs Hz
fig2 = plt.figure(1)
ax2 = fig2.gca()
fig2.suptitle('Frequency domain')
ax2.plot(f_x,y_x,'tab:blue')
ax2.set_ylabel('|FFT| of x signal')
ax2.set_xlabel('Frequency [Hz]')

plt.show()
```



Task 2.3

1. Perform FFT from the example signal, x by using the coding space below.
2. What is the range (min and max) of frequency in FFT?
3. What are the two main frequencies? Are the results as expected?

4. Change the title to 'Frequency domain --Your name--'

Your graph should look like the one below

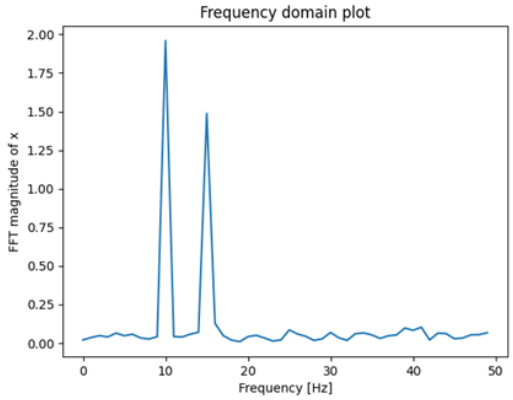
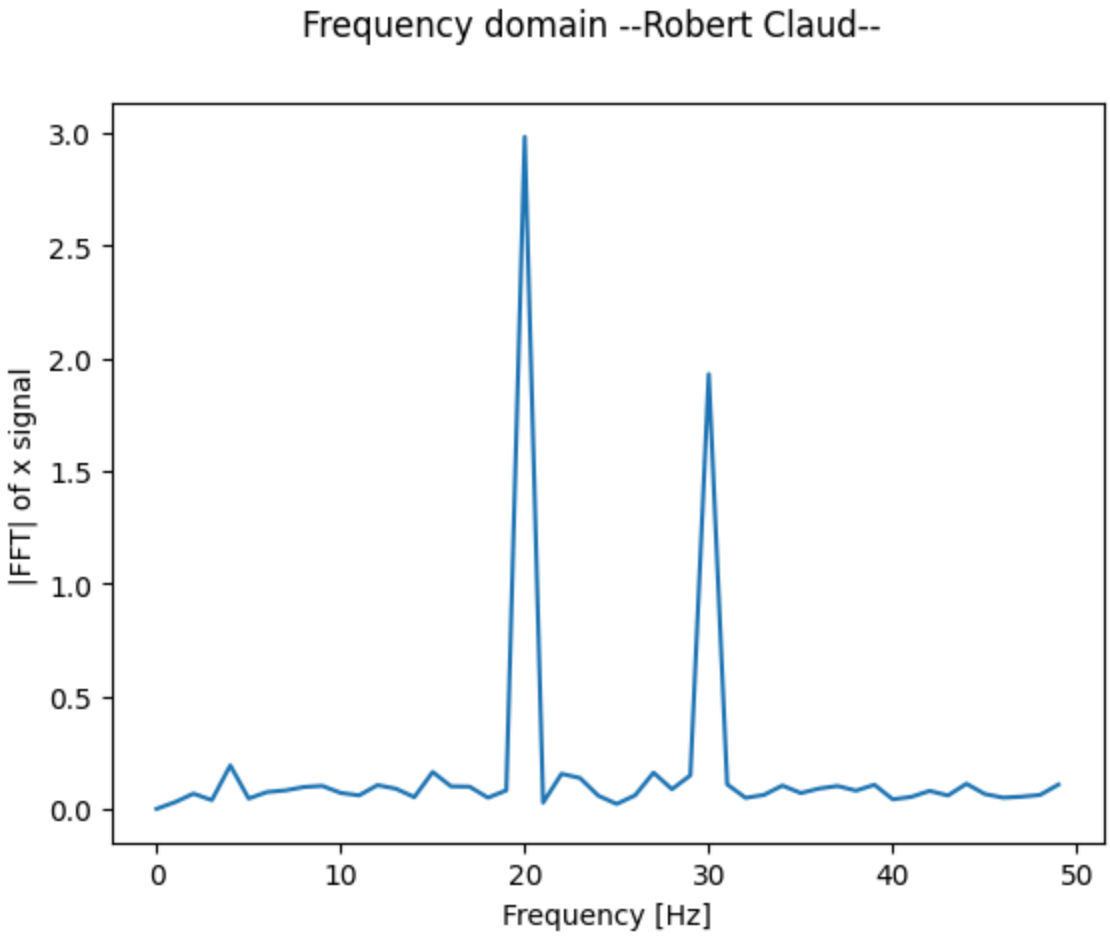


Figure 7 Frequency domain plot of the example signal, x

In [25]:
```
#Insert your code for FFT here:

#First, import fft libraries
from scipy.fft import fft, fftfreq
from scipy.signal import find_peaks

#FFT function.
def get_fft(x, T, N):
    f = fftfreq(N, T)[:N//2]
    y_mag = 2/N * np.abs(fft(x)[:N//2])
    y_mag[0] = 0 # Making 0 Hz frequency component zero
    return f, y_mag

#Output variables
f_x, y_x = get_fft(x, T, N) #Signal x is been used

#Plotting fft vs Hz
fig2 = plt.figure(1)
ax2 = fig2.gca()
fig2.suptitle('Frequency domain --Robert Claud--')
ax2.plot(f_x,y_x,'tab:blue')
ax2.set_ylabel('|FFT| of x signal')
ax2.set_xlabel('Frequency [Hz]')

plt.show()

peaks, _ = find_peaks(y_x, height=max(noise))
print(f"Range of frequency in FFT is {(float(min(f_x)),float(max(f_x)))}")
print(f"The peaks occur at {peaks}")
```




```
Range of frequency in FFT is (0.0, 49.0)
The peaks occur at [20 30]
```

2. What is the range (min and max) of frequency in FFT?
    - The range (min and max) of frequency in FFT is 0 to 49
3. What are the two main frequencies? Are the results as expected?
    - The two main frequencies are 20 and 30. These are the expected results as we have set the signals x_1 and x_2 to have these amplitudes respectively.

Get back to Lab Index Page